Stock Ticker Challenge

Revision Notes · my mistakes, weak areas & practice

Key (designed for black & white printing)	
HIGH **MED** **LOW**	Priority. Spend most revision time on **HIGH**.
✗ / ✓	My wrong code / the correct code
Solid black box	RULE: memorise this
Dashed box	Tanglish ROAST: for fun, nothing new to learn
Bar on the left	Note or code (code: **bold** = keyword, *italic* = comment)
☐ **answer boxes**	Write predictions by hand, then check in the editor

> **Challenge:** Get fresh stock data every 1.5s → call `renderStockTicker` with that data → display it (with a red/green/grey price arrow).

ROAST · Tanglish

Warning: indha notes la konjam roast irukku. Feelings hurt aana, code fix pannu, crying aprom.

Focus Map — Where I Need to Work Hardest

Priority	Weak Area	Mistakes it caused
HIGH	Function reference vs function call (`fn` vs `fn()`)	#1, #2
HIGH	Separation of concerns (fetch vs render vs timer)	#3, #6
MED	Data types (string vs number)	#4
MED	Initial state & edge cases	#5
MED	What an IIFE is + when code runs (module load vs function call)	#7
LOW	Reading the expected output format	#8

ROAST · Tanglish

8 mistakes in one challenge ah? Machi, nee bug report ezhudhura aal. Ippo unakke oru bug report ready!

> **The two HIGH areas matter most.** They are core to React: components receive **data** (props) and only **render** it. If I get these right in plain JS, React will feel natural.

Weak Area 1 — Function Reference vs Function Call **HIGH**

What I did wrong

```
renderStockTicker(getStockData)          // ✗ passed the function itself

stockDisplayName.innerHTML = stockData().name    // ✗ call #1
stockDisplayTime.innerHTML = stockData().time    // ✗ call #2
stockDisplaySymbol.innerHTML = stockData().sym   // ✗ call #3
current = stockData().price                      // ✗ call #4
```

ROAST · Tanglish

Oru stock price ku naalu thadava API call ah? Enna da, stock exchange ah DDoS panna plan ah?

Every `stockData()` call creates a **brand-new object** with a **new random price**. So the four values on screen came from four different "fetches".

The fix

```
const stockData = getStockData()   // ✓ call ONCE → get ONE object
renderStockTicker(stockData)       // ✓ pass the DATA
```

Comparison

	`getStockData`	`getStockData()`
What it is	The function (a recipe)	The result (the cooked dish)
Runs the code?	No	Yes
Type	`function`	`object` (here)
Pass it when...	Someone else should call it later (e.g. `setInterval(fn, 1500)` , `onClick={fn}`)	You want the value now

RULE · REMEMBER THIS

Call once, store in a variable, reuse the variable. If I see `something().x` and `something().y` on nearby lines → **[!]** red flag.

ROAST · Tanglish

Biryani order panni, oru oru piece ku thaniya delivery boy anuppura maari irukku. Oru thadava vaangu, plate la vechu saapdu!

React link

```
<button onClick={handleClick}>   {/* ✓ reference — React calls it on click */}
<button onClick={handleClick()}> {/* ✗ runs immediately on every render */}
```

Weak Area 2 — Separation of Concerns **HIGH**

What I did wrong

`setInterval` was **inside** `renderStockTicker`. The render function was doing three jobs: timing, fetching, and painting.

> **ROAST · Tanglish**
> *Paavam andha function. Timer-um adhuve, delivery-um adhuve, painting-um adhuve. Idhu function illa da, Tamil cinema hero — ellam ore aal pannuvaan!*

The correct shape

```
┌──────────── setInterval (every 1.5s) ────────────┐
│  1. FETCH   →  const data = getStockData()       │
│  2. RENDER  →  renderStockTicker(data)           │
└──────────────────────────────────────────────────┘
renderStockTicker = only paints what it is given ("dumb")
```

Job	Belongs in	NOT in
Timing (`setInterval`)	Outside, top level	render function
Getting data	Inside the interval callback	render function
Updating the DOM	`renderStockTicker`	—
Remembering previous price	Variable **outside** render	local `let` inside render

Knock-on bug (#6)

Once render runs every tick, this code adds a **new** `<img>` **every 1.5s**:

```
const image_el = document.createElement('img')
stockDisplayPriceIcon.append(image_el)   // ✗ stacks up images
```

> **ROAST · Tanglish**
> *1.5 second ku oru image add aagudhu. Oru 10 minute la DOM full ah arrows — idhu stock ticker ah, illa arrow shop ah?*

Fix: **replace** content instead of appending.

```
priceIconEl.innerHTML = `<img src="./svg/${icon}.svg" alt="${icon} price arrow">`
```

RULE · REMEMBER THIS

Render functions should be safe to call 1000 times. If calling it twice causes duplicates or extra timers → it is doing too much.

React link

A React component is exactly a "dumb render function": `props in → UI out`. Timers go in `useEffect`, not in the render body.

Weak Area 3 — Data Types (String vs Number) MED

```
(Math.random() * 3).toFixed(2)   // returns a STRING, e.g. "2.17"
```

Comparison	Result	Why
`"2.10" > "1.95"`	`true`	Works by luck (same length)
`"10.00" > "9.00"`	`false`	Strings compare character by character: `"1" < "9"`
`Number("10.00") > Number("9.00")`	`true`	Number comparison

ROAST · Tanglish

"10" < "9" nu JavaScript solludhu. Nee QA da! Indha edge case ah nee thaan mudhalla pudikkanum. Jira la idha vera yaaraavadhu raise pannaa, maanam poidum!

RULE · REMEMBER THIS

`toFixed()` **is for display. Convert with** `Number()` **before maths or comparisons.** As a QE: always ask "what if the value grows past one digit?"

Weak Area 4 — Initial State & Edge Cases MED

```
let previous = 0   // ✗ first price is always > 0 → always green on tick 1
```

ROAST · Tanglish

First tick laye green arrow? Market open aagradhukku munnaadiye profit ah? Unga stock la semma optimism, machi!

```
let previousPrice = null           // ✓ "no data yet"
if (previousPrice !== null) { ... } // ✓ first tick stays grey
```

RULE · REMEMBER THIS

Ask: "What happens on the very first run?" `0` means "a real value of zero". `null` means "nothing yet". Pick the one that tells the truth.

Weak Area 5 — When Does Code Run? MED

Where the code is	Runs…
Top level of a module	**Once**, when the file loads
Inside a function body	**Every time** the function is called
IIFE `(() => {...})()` inside a function	Every call (but adds noise)

My IIFE was actually fine (it was inside the function). The grader's feedback was misleading.

ROAST · Tanglish
Idhu mattum un thappu illa. Grader thaan confuse aayiduchu. Aana yen da simple ah ezhudhaama style kaatna? Ippo grader-ae kozhambi pochu! But the simpler form is better:

```
export default function getStockData() {
  const now = new Date()
  const hh = String(now.getHours()).padStart(2, '0')
  const mm = String(now.getMinutes()).padStart(2, '0')
  const ss = String(now.getSeconds()).padStart(2, '0')

  return {
    name: 'QtechAI',
    sym: 'QTA',
    price: (Math.random() * 3).toFixed(2),
    time: `${hh}:${mm}:${ss}`,
  }
}
```

RULE · REMEMBER THIS

Prefer plain lines over clever patterns. Clear code is easier for graders, reviewers, and future me.

Deep Dive — What is an IIFE?

ROAST · Tanglish
Code la IIFE ezhudhitu, adhu enna nu theriyaadhu ah? Machi, recipe theriyaama biryani senja maari — taste nalla irundhaalum, epdi vandhuchu nu theriyaadhu!

IIFE = Immediately Invoked Function Expression A function that is **created and called in the same moment**. Define it → run it → done.

Breaking down the syntax

```
(() => {
  console.log('I run immediately!')
})()
```

Piece	Meaning
`() => { ... }`	A normal arrow function (just a recipe, nothing runs yet)
`( ... )` around it	Wraps it so JS treats it as a **value** (an expression)
`()` at the end	**Calls it right now**

Read it as: **"make a function → call it immediately."**

Step by step: normal function vs IIFE

```
// Normal way: 2 steps
const makeTime = () => {
  return new Date().toLocaleTimeString()
}
const time = makeTime()      // step 2: call it

// IIFE way: same thing in 1 step, no name needed
const time = (() => {
  return new Date().toLocaleTimeString()
})()
```

Both give `time` a **string**, not a function.

The `()` at the end changes everything

```
const a = () => 'hello'      // a = a FUNCTION
const b = (() => 'hello')()  // b = 'hello' (a STRING)

console.log(typeof a)   // "function"
console.log(typeof b)   // "string"
```

> This is **Weak Area 1 again** (reference vs call)! An IIFE is just a function call stuck right onto the function.

What my code actually did

```
time: (() => {
  const now = new Date()
  ...
  return `${hh}/${mm}/${ss}`
})()
```

- The IIFE runs **while the object is being built** → `time` becomes a string like `"14:32:05"` .
- It sits inside `getStockData()` → runs **every time** `getStockData()` is called → fresh time .

IIFE: once or every time? It depends on WHERE it is

Where the IIFE lives	How often it runs
Top level of a file	**Once**, when the file loads
Inside a function	**Every call** of that function
Inside a `setInterval` callback	**Every tick**

```
// ✗ Top level: runs ONCE
const loadTime = (() => new Date().toLocaleTimeString())()
export default function getStockData() {
  return { time: loadTime }    // same time forever
}

// ✓ Inside the function: runs EVERY call
export default function getStockData() {
  return { time: (() => new Date().toLocaleTimeString())() }
}
```

When are IIFEs actually useful?

Use case	Example
Keep variables private (old-school, before modules)	Avoid polluting global scope
Run `async` code at the top level (older setups)	`(async () => { await fetch(...) })()`
Compute a value with several steps, inline	Complex value inside JSX / an object
My challenge?	Not needed. Plain lines are clearer

ROAST · Tanglish
Simple ah 4 line ezhudha vendiya idathula IIFE potta. Bus stand ku pogha helicopter book panna maari, machi!

RULE · REMEMBER THIS

IIFE = function + `()` at the end = runs now. Where it sits decides whether it runs **once** or **every time**. If plain lines do the job, skip the IIFE.

React link

You'll see IIFEs in React when people need multi-step logic inside JSX:

```
<p>
  {(() => {
    if (price > 2) return 'High'
    if (price > 1) return 'Medium'
    return 'Low'
  })()}
</p>
```

Usually it's cleaner to compute the value **above the** `return` in a variable instead.

Weak Area 6 — Reading the Expected Output LOW

`hh/mm/ss` vs `hh:mm:ss` — check the exact format the challenge or UI expects before submitting.

ROAST · Tanglish
Time ah date maari slash pottu ezhudhitta. 14/32/05 — idhu enna thethi? 14th month 32nd date ah?

Final Correct Solution

ROAST · Tanglish

Idho, ippo dhaan proper ah irukku. Paaru, evlo clean ah irukku — idha first time laye ezhudhirundha naan roast panna vendiyadhilla!

```
// index.js
import getStockData from './fakeStockAPI.js'

let previousPrice = null

function renderStockTicker(stockData) {
  const { name, sym, price, time } = stockData

  document.getElementById('name').textContent = name
  document.getElementById('symbol').textContent = sym
  document.getElementById('price').textContent = price
  document.getElementById('time').textContent = time

  const currentPrice = Number(price)
  let icon = 'grey'
  if (previousPrice !== null) {
    if (currentPrice > previousPrice) icon = 'green'
    else if (currentPrice < previousPrice) icon = 'red'
  }

  document.getElementById('price-icon').innerHTML =
    `<img src="./svg/${icon}.svg" alt="${icon} price arrow">`

  previousPrice = currentPrice
}

setInterval(() => {
  const stockData = getStockData()
  renderStockTicker(stockData)
}, 1500)
```

Practice Exercises (do these in the editor)

ROAST · Tanglish
Notes padichitu "puriyudhu" nu solli close panna koodaadhu. Editor open pannu, type pannu. Copy-paste panna, adhuvum oru bug dhaan!

Ex 1 — Reference vs call (HIGH)

Predict the output, then run it:

```
function roll() { return Math.ceil(Math.random() * 6) }
console.log(roll().toString(), roll().toString())  // same or different?
const r = roll()
console.log(r, r)                                  // same or different?
console.log(typeof roll, typeof roll())
```

My answer / notes ☐ Done

Ex 2 — Dumb render (HIGH)

Build a clock: `getTime()` returns `{ hh, mm, ss }`, `renderClock(time)` only updates the DOM, and `setInterval` lives outside both. Then call `renderClock` 5 times in a row: does anything duplicate? (It should not.)

My answer / notes ☐ Done

Ex 3 — Spot the stacking bug (HIGH)

Write a render function that uses `append()` for a badge. Put it in a 1s interval and watch the DOM in DevTools. Then fix it with `innerHTML` or `replaceChildren()`.

My answer / notes ☐ Done

Ex 4 — String comparison trap (MED)

```
console.log("10" > "9", 10 > 9, "2.5" > "12.0", Number("2.5") > Number("12.0"))
```

Explain each result in one line.

My answer / notes ☐ Done

Ex 5 — First-run edge case (MED)

Write a temperature tracker that shows ↑/↓/– vs the previous reading. Make sure the very first reading shows –.

My answer / notes ☐ Done

Ex 6 — IIFE basics (MED)

Predict each output, then run it:

```
const f = () => 42
const g = (() => 42)()
console.log(typeof f, typeof g)

const once = (() => Math.random())()
function every() { return (() => Math.random())() }
console.log(once, once)        // same or different?
console.log(every(), every())  // same or different?
```

Then rewrite this IIFE as normal lines with a variable:

```
const greeting = (() => {
  const hour = new Date().getHours()
  return hour < 12 ? 'Good morning' : 'Good evening'
})()
```

My answer / notes ☐ Done

One-Line Takeaways

1. **HIGH** **Call once, store, reuse.** `fn` = recipe, `fn()` = result.
2. **HIGH** **Timer outside, render dumb.** Render only paints the data it is given.
3. **HIGH** **Render must be safe to repeat.** Replace, don't append.
4. **MED** `toFixed()` **gives a string.** Use `Number()` before comparing.
5. **MED** **Think about the first run.** Use `null` for "no value yet".
6. **MED** **IIFE = function +** `()` **= runs now.** Top-level = once; inside a function = every call.
7. **LOW** **Read the expected output format carefully.**

ROAST · Tanglish

Final roast: Daily vera aal code la bugs kandupidikkura, aana un code la 8 bugs. "Doctor-ku udambu sari illa" moment! But seriously, nee semma close ah irundha. Fix pannitu next challenge ku po, machi. You got this!